

Documento de Planejamento de Testes e Estratégia de QA - Squad Last but not least

Identificação do Plano

Plano de Testes ID: Projeto 2 - Last but not least

Projeto: Projeto 2

Versão/Teste: Versão 1

Responsáveis pelo Plano: Ana Leticia de Araújo e Paulo Renato Junco Da Silva Braga

Data de Criação/Atualização: 16/01/2026

Introdução e Objetivo

Contexto do Projeto

Este Plano de Testes estabelece as metodologias, ferramentas e diretrizes que serão utilizadas para validar a qualidade do **Projeto 2**. Em um cenário de desenvolvimento ágil, este documento serve como guia para garantir que cada funcionalidade entregue esteja em conformidade com as expectativas dos usuários finais e os requisitos de negócio.

Este documento descreve a estratégia de testes para o **Projeto 2**, visando garantir que as entregas atendam aos critérios de qualidade definidos, reduzindo riscos e garantindo a satisfação do usuário final.

Objetivo Geral

O foco central aqui é estabelecer um fluxo de validação consistente para antecipar eventuais bugs. Isso diminui a ocorrência de falhas do sistema em produção e mantém a operação rodando sem sustos.

Objetivos Específicos

- **Garantia da Qualidade (QA):** O foco é garantir que a entrega final esteja alinhada ao que a Squad e o responsável pelo produto estabeleceram como meta.
- **Mitigação de Riscos:** Busca-se encontrar gargalos técnicos ou falhas de raciocínio lógico bem antes de o código ser liberado para o público..
- **Feedback Rápido:** Baseando-se na pirâmide de testes, define-se um jeito de testar que dê um retorno rápido aos desenvolvedores sobre a saúde do sistema, ganhando tempo no dia a dia.

- **Satisfação do Usuário:** O propósito é assegurar que quem usar o sistema tenha uma experiência fluida, segura e sem erros que travem a navegação.

Escopo do Planejamento (O que será testado?)

Este trecho delimita as fronteiras do trabalho de QA, garantindo que o esforço se concentre no que é vital para o funcionamento do negócio no momento.

Funcionalidades em Foco: (Liste aqui as principais funções do curso, ex: *Login*, *Cadastro*, *Busca*).

Nesta fase, a atenção se volta para as funções que garantem que o sistema cumpra sua tarefa básica:

Módulo de Autenticação (*Login*):

- Conferência de acessos com dados certos e errados.
- Verificação de como o sistema lida com campos vazios ou e-mails digitados incorretamente.
- Teste para ver se a pessoa continua conectada após entrar no site.

Gestão de Usuários (*Cadastro*):

- Validação da criação de novas contas e garantia de que dados obrigatórios não fiquem de fora.
- Validação de regras de negócio (ex: impedimento de cadastros repetidos com o mesmo e-mail).

Módulo de Busca e Filtros:

- Precisão dos resultados retornados de acordo com os termos pesquisados.
- Avaliação de como o site se comporta quando não encontra nenhum produto (*Empty States*).

Interface e Navegação (UI):

- Garantia de que os elementos visuais (botões, *links* e menus) funcionam ao serem clicados e levam para as telas corretas.

Requisitos Não-Funcionais e Exclusões (Fora do Escopo)

Para manter o foco na estabilidade funcional nesta fase inicial do projeto, os seguintes itens **não** serão abordados:

- **Testes de Carga e Performance:** Não será validado o comportamento do sistema sob estresse de servidor ou alto volume de acessos simultâneos.
- **Testes de Segurança Ofensiva (Pentest):** Verificações de vulnerabilidades complexas de segurança (como ataques XSS ou injeção de SQL) estão fora deste ciclo.
- **Compatibilidade com Legados:** O sistema não apresentará suporte para versões obsoletas (como Internet Explorer), sendo testado somente em navegadores modernos

(Chrome/Edge/Firefox).

- **Testes de Acessibilidade:** Validações de conformidade com normas WCAG não serão executadas neste primeiro *sprint*.

Níveis de Teste (Baseado na Pirâmide de Testes)

- **Testes de Unidade:** Foco em lógica de negócio isolada (desenvolvedores).
- **Testes de Integração/API:** Validar a comunicação entre banco de dados e sistemas (mais estáveis e rápidos que UI).
- **Testes de UI/E2E:** Validar fluxos críticos do usuário. *Nota estratégica: Manter uma suíte enxuta para evitar a alta manutenção e latência observada em ferramentas como Selenium.*

Estratégia de Testes

Para garantir que a entrega seja segura, o plano segue estas frentes:

- **Testes Manuais Exploratórios:** Uso livre do sistema para encontrar erros visuais ou de fluxo que um robô poderia ignorar.
- **Testes de API:** Conferência técnica das mensagens do servidor (*status code*, JSON) para ver se a integração entre as partes está funcionando bem.
- **Testes Funcionais:** Execução de roteiros específicos para confirmar que as regras do negócio estão sendo seguidas.
- **Testes de Regressão:** Repetição das verificações básicas a cada mudança, para ter certeza de que o código novo não quebrou o que já funcionava.

Ambiente e Ferramentas

Aqui estão detalhados o local e os recursos escolhidos para apoiar o ciclo de vida de testes (STLC).

Ambiente de Teste:

Homologação (UAT): Ambiente principal onde serão executados os testes funcionais e de aceitação. É um espelho do ambiente de produção, garantindo que os testes reflitam o comportamento real do sistema.

Staging: Utilizado para testes de integração final e validação de *deploy*. Este ambiente permite verificar se as novas alterações não afetam as integrações existentes em um cenário pré-lançamento.

Stack de Automação e Linguagem:

Linguagem Python: Escolhida pela clareza e pela facilidade em automatizar tarefas e organizar dados.

Framework Pytest: Ferramenta que organiza e executa as automações, permitindo desde verificações simples até configurações mais avançadas.

API Testing:

Postman / Insomnia: Ferramentas usadas para desenhar e rodar os testes de API, conferindo dados, códigos de erro e tempo de resposta. Serão validados:

- *Status Codes* (200 OK, 201 Created, 400 Bad Request, etc.).
- Contrato e estrutura do JSON de resposta.
- Tempo de resposta (Latência) e consistência dos dados retornados.

Gestão e Documentação:

Jira Software: Ferramenta central para a gestão de defeitos. Cada *bug* identificado será reportado com evidências (prints ou logs), severidade e prioridade, seguindo o fluxo de status: *Aberto* → *Em Correção* → *Re-teste* → *Fechado*.

Confluence: Centralização de toda a documentação estratégica, planos de teste e relatórios de execução, facilitando a transparência e colaboração (mesmo em execuções individuais).

Critérios de Aceite (*Definition of Done*)

A *Definition of Done* (DoD) é o acordo que garante que todos os membros da equipe (e stakeholders) entendam o que foi finalizado com qualidade. Para este projeto, uma funcionalidade só será considerada "Pronta" após atender aos seguintes requisitos:

Testes Manuais e Exploratórios Concluídos

- **Validação de Interface (UI):** O layout deve estar funcional e seguindo o desenho planejado.
- **Usabilidade:** O fluxo deve ser intuitivo, sem travamentos ou comportamentos estranhos para o usuário final.
- **Testes de Borda (*Edge Cases*):** Realizar testes manuais em cenários que a automação pode não cobrir inicialmente, como comportamentos específicos de navegação.

Automação de Caminhos Felizes (*Happy Paths*)

- **Cobertura de Fluxo Principal:** O fluxo de sucesso da funcionalidade (ex: realizar uma compra do início ao fim sem erros) deve estar 100% automatizado em Python/Pytest.

- **Estabilidade:** Os testes automatizados devem ser executados com sucesso em ambiente de Homologação, servindo como uma rede de segurança para futuras regressões.
- **Integração:** Validação de que o novo código automatizado não quebrou funcionalidades existentes.

Gestão de Defeitos e Severidade

- **Bloqueios Críticos:** Não será permitida a subida para Produção de funcionalidades com *bugs* de severidade Alta (funcionalidade comprometida) ou Crítica (sistema indisponível ou perda de dados).
- **Bugs de Baixa/Média Severidade:** Devem ser documentados no Jira. A decisão de correção imediata ou inclusão no próximo backlog deve ser alinhada com o P.O. (*Product Owner*).
- **Evidências:** Todos os bugs encontrados devem possuir logs, prints ou vídeos anexados para facilitar a correção pelo time de desenvolvimento.

Documentação e Evidências

- **Atualização do Confluence:** Este plano de testes e os resultados das execuções devem estar devidamente atualizados.
- **Massa de Dados:** Caso tenham sido criados novos usuários ou produtos para o teste, estes devem ser catalogados para uso futuro.

Cronograma e Entregáveis

Timeframes:

- Preparação do ambiente: [data]
- Execução das rodadas de teste: [datas]
- Relatórios de defeitos: Fluxo contínuo durante a execução.

Entregáveis: Casos de teste completos, relatórios de execução no Jira e sumário executivo de testes no Confluence.

Riscos e Mitigações

Plano de Riscos

Abaixo estão os pontos que podem impactar o cronograma de testes:

- **Ambiente de Teste Instável:** Quedas no servidor de homologação podem impedir a execução dos testes.
- **Atraso no Desenvolvimento:** Caso as funcionalidades não sejam entregues no prazo, o tempo de QA será reduzido.

- **Massa de Dados Insuficiente:** Falta de usuários ou produtos cadastrados para realizar os testes de fluxo completo
- **Mudança de Requisitos:** Alterações de última hora no que foi planejado podem gerar retrabalho nos casos de teste.

Plano de Mitigação

Para cada risco mapeado, definimos as seguintes ações preventivas e corretivas:

- **Para Ambiente de Teste Instável:** Configurar um ambiente local de contingência ou utilizar **Mocks/Stubs** para simular serviços externos e APIs, permitindo que os testes de lógica continuem mesmo se o servidor principal estiver fora do ar.
- **Para Atraso no Desenvolvimento:** Adotar a prática de *Shift Left Testing*, iniciando a escrita dos casos de teste e a validação de documentos de requisitos antes mesmo do código estar pronto. Caso o atraso persista, priorizar a execução apenas dos **Testes de Fumaça** (*Smoke Tests*) e caminhos críticos.
- **Para Massa de Dados Insuficiente:** Desenvolver scripts de automação para inserção de dados via banco ou API (*Data Seeding*) e criar uma planilha de massa de dados padrão que possa ser reutilizada e reiniciada rapidamente.
- **Para Mudança de Requisitos:** Estabelecer um canal de comunicação direta com o P.O. (*Product Owner*) para alinhamento imediato. Realizar uma análise de impacto rápida para atualizar apenas os casos de teste afetados, evitando o retrabalho na suíte completa.

Comunicação e Aprovação

Os resultados serão comunicados via reuniões de Daily e relatórios semanais por e-mail para os envolvidos na Squad. O início oficial das rodadas de teste depende da aprovação deste plano pelo responsável pelo produto e pela liderança técnica.